

LOOP Instructions (8086)

What is a LOOP instruction?

LOOP is a compact way to repeat a block of code a fixed number of times. It is essentially:

$$CX = CX - 1 \quad \text{then} \quad \text{if } CX \neq 0 \text{ jump to a label}$$

Key idea: CX works as the **default loop counter**. Every iteration decreases CX by 1.

1) LOOP: Syntax and Core Behavior

Syntax

```
LOOP short_label
```

Execution steps (what the CPU does)

When the CPU executes `LOOP label`, it performs:

1. **Decrement** CX by 1.
2. **Check** whether CX is zero.
3. If $CX \neq 0 \rightarrow$ **jump** to label.
4. If $CX = 0 \rightarrow$ continue with the **next instruction**.

Important notes

- LOOP always uses **CX**. If you want another register to act as counter, you must manually move it into CX.
- If CX starts as 0, LOOP will decrement first (to `FFFFh`) and the loop will run many times. So **initialize CX carefully**.
- `short_label` means the target label is in a short jump range (common in typical assembly blocks).

2) Mental Model (Easy Way to Remember)

Think of LOOP as

“Count down in CX until it becomes 0.”

$$\underbrace{CX}_{\text{counter}} \rightarrow CX - 1 \rightarrow \begin{cases} \text{jump back} & \text{if } CX \neq 0 \\ \text{exit loop} & \text{if } CX = 0 \end{cases}$$

3) Example A: Simple Counting Loop

Goal: execute a body exactly **5 times**.

Listing 1: Example A: loop exactly 5 times

```
1 MOV CX, 5 ; set loop counter to 5
2 L1:
3   ; ---- loop body ----
4   NOP ; (placeholder instruction)
5   ; -----
6 LOOP L1 ; CX = CX-1, jump if CX != 0
```

Trace (iteration-by-iteration)

- Start: CX = 5
 - After 1st LOOP: CX = 4 → jump
 - After 2nd LOOP: CX = 3 → jump
 - After 3rd LOOP: CX = 2 → jump
 - After 4th LOOP: CX = 1 → jump
 - After 5th LOOP: CX = 0 → exit
- So the body runs **exactly 5 times**.

4) Example B: Process Two Blocks Using String Ops

This is a cleaned, well-commented version of the idea shown in the lecture: repeat block operations a fixed number of times using CX and LOOP.

Listing 2: Example B: add two word arrays and store result (100 times)

```
1 _DATA SEGMENT
2 BLOCK1 DW 100 DUP(?) ; 100 words
3 BLOCK2 DW 100 DUP(?) ; 100 words
4 _DATA ENDS
5
6 _CODE SEGMENT
7 STARTUP:
8   MOV AX, DS
9   MOV ES, AX ; ES overlaps DS (same segment)
10  CLD ; DF=0 => auto-increment SI/DI
11
```

```

12  MOV CX, 100 ; loop counter = 100 iterations
13  MOV SI, OFFSET BLOCK1
14  MOV DI, OFFSET BLOCK2
15
16  L1:
17  LODSW ; AX <- [DS:SI], then SI += 2
18  ADD AX, ES:[DI] ; AX = AX + BLOCK2[i]
19  STOSW ; [ES:DI] <- AX, then DI += 2
20  LOOP L1 ; repeat until CX becomes 0
21
22  _CODE ENDS
23  END STARTUP

```

Why this loop is efficient

- LODSW and STOSW automatically update SI and DI.
- CX tracks the iteration count (100).
- One instruction LOOP does both counter update and conditional jump.

5) Conditional LOOPS: LOOPE/LOOPNE (LOOPZ/LOOPNZ)

Sometimes you want a loop to continue not only based on CX, but also based on a condition (usually related to the **Zero Flag**, ZF).

Key Definitions

- **LOOPE / LOOPZ**: jump if CX $\neq$ 0 and ZF = 1 (equal / zero condition holds).
- **LOOPNE / LOOPNZ**: jump if CX $\neq$ 0 and ZF = 0 (not equal / not zero condition holds).

These loops exit when:

- counter ends (CX = 0), **or**
- the equality condition breaks (ZF changes).

Syntax

```

LOOPE short_label / LOOPZ short_label
LOOPNE short_label / LOOPNZ short_label

```

Example C: Stop when mismatch occurs OR when CX runs out

Suppose we compare two bytes each iteration and want to keep looping while they are equal. (The compare sets ZF.)

Listing 3: Example C: loop while equal AND CX != 0

```

1  MOV CX, 20 ; at most 20 comparisons
2  Lcmp:

```

```

3 ; assume AL and [DI] are compared
4 CMP AL, [DI] ; sets ZF = 1 if equal
5 INC DI
6 LOOPE Lcmp ; jump if (CX!=0) AND (ZF=1)
7 ; exits if mismatch (ZF=0) OR CX==0

```

Common confusion

LOOPE still decrements **CX** like normal **LOOP**. The difference is: it also checks **ZF**. So it's really:

$$CX \leftarrow CX - 1, \quad \text{jump if } (CX \neq 0) \wedge (ZF \text{ matches})$$

6) Quick Summary

One-page recap

- **LOOP** label: $CX--$; if $CX \neq 0$ jump.
- **LOOPE/LOOPZ** label: $CX--$; jump if $CX \neq 0$ and $ZF=1$.
- **LOOPNE/LOOPNZ** label: $CX--$; jump if $CX \neq 0$ and $ZF=0$.
- Always remember: **CX** is the counter, and conditional loops also depend on **ZF**.